

Interview Preparation Guide — AI/ML Engineer

1. Core ML Concepts

Bias-variance trade-off, overfitting/regularization, and evaluation metrics (precision/recall/F1, ROC-AUC) under class imbalance.

2. Deep Learning and NLP

Transformer architecture basics (self-attention, positional encoding), encoder vs. decoder-only models, embeddings, fine-tuning vs. RAG, and prompt engineering trade-offs.

3. Practical/Applied Focus

Most local roles are applied rather than research-focused: working with LLM APIs, vector databases, embedding-based search, and integrating a model into a production backend.

Scenario-Based Questions

Scenario-based questions test applied judgment rather than textbook recall — interviewers are usually more interested in your reasoning process than the exact final answer.

Scenario: A RAG chatbot you built is confidently giving wrong answers on questions it should know from its documents. Walk through how you'd debug whether the problem is retrieval or generation.

Scenario: Your model performs great in offline evaluation but users complain it's unhelpful in production. What could explain the gap, and how would you investigate?

Scenario: You need to add a new document category to an existing RAG system without breaking answer quality on the categories that already work well. How do you approach this safely?

Questions by Experience Level

The bar for the same topic area rises sharply by level. Below is a representative question for an AI/ML Engineer at each tier, showing how depth of expected answer changes — not just difficulty of the question itself.

Intern: Q: What is overfitting, and name one way to reduce it.

Entry-Level: Q: Explain what an embedding is and why cosine similarity is commonly used to compare them.

Associate Engineer: Q: Why does the embedding dimension need to match the vector database's index configuration, and what happens if it doesn't?

Engineer: Q: Design a retrieval-then-rerank pipeline and explain why you wouldn't just increase top_k instead of adding a reranker.

Senior: Q: How would you evaluate whether a RAG system's answers are actually grounded in retrieved context versus the LLM's own memorized knowledge?

Principal: Q: How would you decide, for a company with a dozen internal RAG use cases, whether to standardize on one shared retrieval infrastructure or let each team choose its own?